



Audio tömörítő egység megvalósítása FPGA-val

Diplomatervezés 1 zárójegyzőkönyv

2025/26. 2. félév

Nyiri Levente (ULYHQH)

Villamosmérnök szakos hallgató

MSc FPGA alapú rendszerek mellékspecializáció

Konzulens:

Szántó Péter

(Mesterséges Intelligencia és Rendszertervezés Tanszék)

Feladatkiírás

Az AMD Ultrascale+ MPSoC FPGA-k használata számos multimédia feldolgozást igénylő alkalmazásban kedvező opció.

Míg a video tömörítésre az eszköz dedikált hardveregységet tartalmaz, addig az audio kezelése elsősorban az általános célú ARM processzoros alrendszeren oldható meg. Ez azonban fogyasztás, illetve CPU terhelés szempontjából nem feltétlenül optimális.

A feladat célja, az alkalmazások jelentős részében előkerülő audiotömörítésre egy, az FPGA általános erőforrásait felhasználó hardver gyorsító kifejlesztése, valamint ennek összehasonlítása a tisztán szoftveres megoldással.

Tartalomjegyzék

1. Vázlat	1
-----------	---

1. Vázlat

A hibrid filterbank előállításához a szabványban előírt módon járok el. Megvárom amíg 512 minta beérkezik, majd ebbe a tömbbe beshiftelek 32 új mintát, és megkezdem a feldolgozást. Először is ablakolni kell, ehhez egy táblázatban megadták a szükséges együtt-hatókat. Mivel ezek az együtt-hatók mind 1-nél kisebb számok, ezért némi átalakítást kell végezni rajtuk, mivel Verilogba egész számokra van szükségünk. Az eljárás a következő: először is egy szöveges file-ba kiírtam és úgy formáztam, hogy minden sorban csak egy bejegyzés legyen. Ezután egy C program segítségével beolvastam ezeket egy tömbbe, megszoroztam 2^{31} -el és hex-be átváltva kiírtam egy másik, .mem file-ba, amit majd a verilog modulomban tudok használni.

A negatív számokat is unsigned long változóban tárolom, ami nem probléma, mert kettes komplement számábrázolásban lesz, csak emlékezni kell rá. Mivel 10 digitos számok az együtt-hatók (1 egészrész, 9 tört), ezért 32 bites változóban kell majd tárolni, mivel 2^{31} -el kellett szorozni, hogy egészek legyenek. Néhány számot visszaellenőrizve meggyőződtem az átalakítás helyességéről (a második bejegyzés például -0.000000477 , ami átváltása után FFFFC00 lett, ezt visszaváltva decimálisba -1024 -et kapunk, ami kerekítési hibával $-0.000000477 * 2^{31}$). Ezeket az együtt-hatókat egy 512x32-es ROM-ban tárolom.

A számításnál a flowchart alapján (1. ábra) be kell shiftelni 32 új mintát, a legrégebbi 32-t pedig ki kell shiftelni. Ezután megadott együtt-hatókkal ablakolni kell, az így kapott értékeket 64 elemenként szummázni, és az ezekből az értékekből alkotott mátrixot szorozni egy másik együtt-ható mátrixsal, amelyhez megadták a képletet a szabványban, hogy az együtt-hatóit kiszámoljuk. A minták kezelését FPGA-ban erőforrás szempontjából hatékony, ha egy BRAM-ban megvalósított cirkuláris bufferben tesszük. Első lépésben feltöltöm 512 bejövő mintával a RAM-ot. Ekkor a legrégebbi mintáim a 0-31 címeken helyezkednek el, így az első alkalommal, amikor új mintákat olvasok be, a 0-31 címekre helyezem őket. Egy változóban tárolom az olvasáshoz az offsetet, amely minden 32 minta shiftelése után 32-vel növelek. A minták olvasását mindig ettől az offsettől kezdem. A címszámlálót a legújabb mintától kezdve csökkentem, 511-ről kezdve amíg el nem éri a 0-t, mivel a legújabb minta van a legrégebbi címre beírva, visszafele kell kiolvasni őket.

BEGIN



for i=511 down to 32 do
 $X[i] = X[i-32]$



for i=31 down to 0 do
 $X[i] = \text{next_input_audio_sample}$



Window by 512 Coefficients
Produce Vector Z
for i=0 to 511 do $Z_i = C_i * X_i$



Partial Calculation
for i=0 to 63 do $Y_i = \sum_{j=0}^7 Z_i + 64j$